

Subchat Trees: Architecture Overview

a Interactive Branching Interface

User-driven branching from selected parent text

Selection-Based Branching

- User highlights a text span in the parent node
- A focused child subchat is created from that span
- Follow-up prompt F is attached to guide the child
- Parent context is inherited once at creation

Tree Navigation View

- Root node represents the main conversation
- Child nodes represent focused follow-up threads
- Depth encodes lineage $r \rightarrow \dots \rightarrow n$
- Sibling branches remain context-isolated

Messenger-Style Interaction

— Familiar chat interface with in-place branching and focused follow-ups

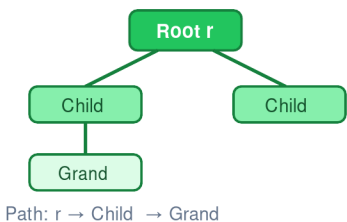
Branching preserves the parent thread while opening a new child context.

(a) User interface with text-selection-driven subchat creation

b Hierarchical Conversation Forest

Forest = $\{T_1, T_2, \dots, T_p\}$ of rooted conversation trees

Tree $T = (N, E, r, B, V)$



Node Semantics

- Each node stores title, parent, children, and path
- Each node owns a local buffer B_n
- Follow-up nodes store selected-text context
- Child nodes copy parent state once at creation
- Subsequent turns remain local to that branch

Def. 1. Each node $n \in N$ has local state, parent linkage, and branch-specific memory. Siblings do not share context; inheritance occurs only at child creation.

(b) Hierarchical tree structure for conversation organization

creates

LLM

isolates

responds with prior knowledge

R (retrieved)
(on-demand)

$B_n + S_{sum}$
(always)

LLM
Response

d On-Demand Retrieval Pipeline

Long-term retrieval from the vector archive when local context is insufficient

Retrieval trigger: The system decides whether archived context is needed for the current query. If not triggered, generation proceeds using node-local memory only.

Phase 1 Decompose

Generate q plus up to $n \leq 8$ focused reformulations of the user query

Phase 2 Search

Embed each sub-query with all-mpnet-base-v2 and search ChromaDB ($N = 20$ candidates/query)

Phase 3 Re-rank

Deduplicate the candidate pool
Re-rank against the original query and keep top-k anchors

Phase 4 Expand

Expand each anchor into a turn-based window $\pm \delta$ ($\delta = 2$) to form coherent context blocks R

Embedding model: all-mpnet-base-v2, $d = 768$

Output: a small set of coherent retrieved blocks R for prompt assembly

queries

(d) Multi-query retrieval pipeline for on-demand long-term memory access

c Three-Tier Node Memory

Per-node short-, medium-, and long-term context

Tier 1 Short-Term: Circular Buffer B_n

FIFO deque · C messages (default 15) · Per-node isolated

Tier 2 Medium-Term: Rolling Summary S_n

$S_n = \text{LLM}(S_{\{n-1\}} \oplus M_{buffer})$ · Triggered at C, 2C, 3C, ...

Tier 3 Long-Term: Vector Archive V

Immediate vector indexing with node_id, role, timestamp, and turn number

Prompt assembly: local buffer and rolling summary are always available
Retrieved blocks R are appended only when retrieval is triggered for the current query.

Context Inheritance (Eq. 4):

At child creation: selected-text prompt + parent summary + parent buffer copy
One-time inheritance · No sibling sharing · No backward propagation to the parent

(c) Local buffer with fixed-size memory and rolling summarization